

ESTRUCTURA DE DATOS

UNIDAD I



Material de lectura

MATERIAL DE CLASE N° 2
CONCEPTOS Y PRACTICAS
BÁSICAS

INTRODUCCION

En informática las Estructuras de Datos son una forma de organizar los datos dentro de la computadora para utilizarlos de manera eficiente. Sus aplicaciones van desde las bases de datos hasta la Internet misma. Además, son claves para crear algoritmos y sistemas de software.

Con el desarrollo de la presente unidad se espera que el cursante sea capaz de:

- ✓ Identificar tipos y estructura de datos.



MATERIALES DE LECTURA DE LA UNIDAD

- Ver material de lectura de conceptos básicos :

[UNIDAD I - CONCEPTOS BASICOS](#)

- Bibliografía sugerida:

[VER LIBRO](#)

Clasificación de Estructuras de Datos

- Estructuras de Datos
 - Estáticas
 - Tipos Primitivos o básicos
 - Arreglos
 - Unidimensionales
 - Bidimensionales
 - Multidimensionales
 - Dinámicas
 - Pilas
 - Colas
 - Listas
 - Árboles
 - Grafos

Tipos de Datos Primitivos

- Numéricos
 - Entero
 - Real
- No numéricos
 - Booleanos
 - Alfanuméricos



Arreglos

- Son variables que permiten múltiples valores de un mismo tipo
- Tienen:
 - Nombre
 - Cantidad de Índices
 - Tamaño de los Índices

Arreglo Unidimensional

Finito: Contiene N elementos.
Homogéneo: del mismo tipo.
Ordenado: con una posición.
Referenciado: con un índice.

PRIMERO	SEGUNDO	TERCERO	CUARTO	QUINTO	SEXTO	SÉPTIMO	...	ENÉSIMO
C_1	C_2	C_3	C_4	C_5	C_6	C_7	...	C_N
i_0	i_1	i_2	i_3	i_4	i_5	i_6	...	i_{N-1}

Fuente: <https://posmarketpro.wordpress.com/capitulo-6-arreglos/>

Registros

- Son variables que permiten conjuntos de datos de diferentes tipos
 - Tienen
 - Nombre del registro
 - Miembros con nombre y tipo
- Ejemplo:
- Registro: Estudiante
 - Nombre: Texto
 - Edad: Número Entero
 - FechaNacimiento: Fecha
 - Fin Registro

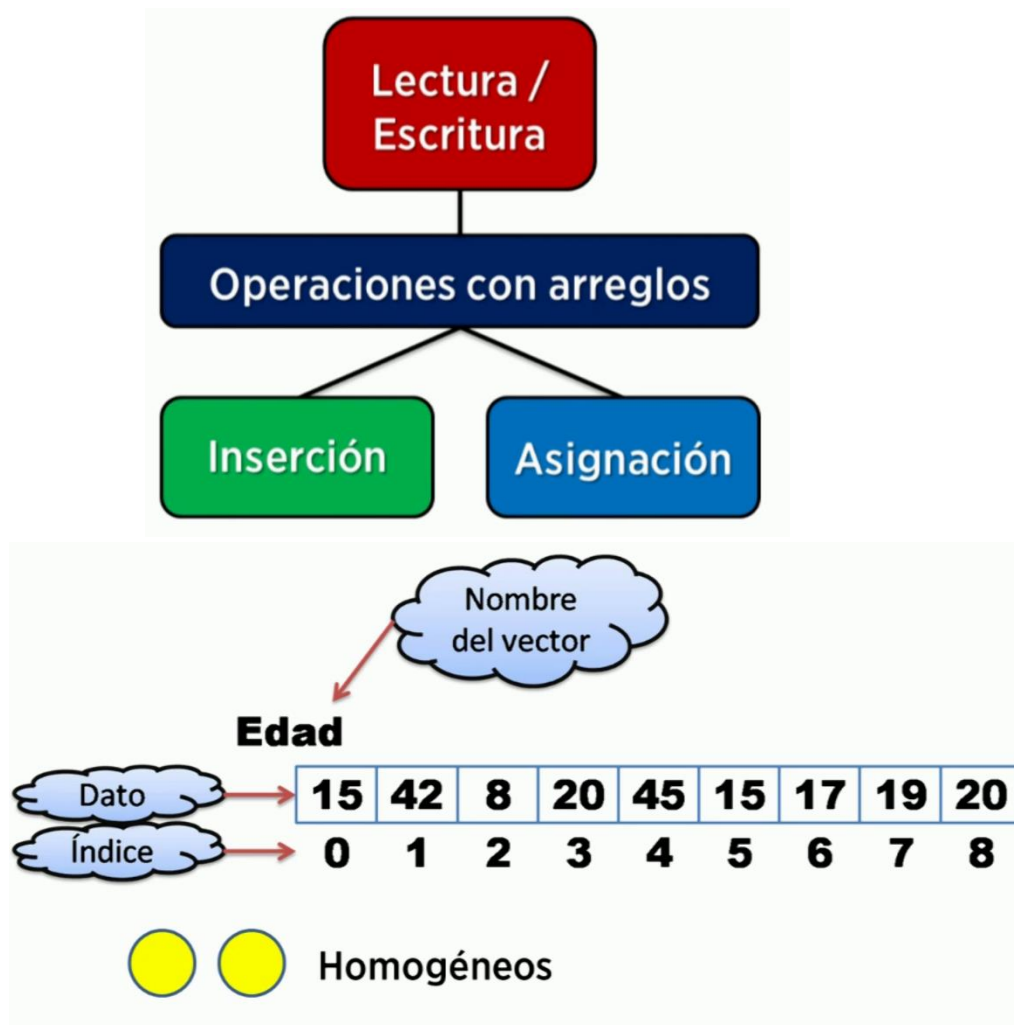
Las Estructuras de datos estáticas: Son aquellas cuyo tamaño se define en tiempo de compilación y no puede modificarse durante la ejecución del programa.

- Ejemplos típicos:

- Arreglos (vectores): Colecciones de elementos homogéneos (mismo tipo de dato) almacenados en posiciones contiguas de memoria.
- Matrices: Extensión de los arreglos a dos o más dimensiones.

Características principales:

- Acceso directo a los elementos mediante índices.
- Memoria fija y predefinida.
- Mayor eficiencia en acceso, pero menor flexibilidad frente a estructuras dinámica



Estructura de un Algoritmo para Gestión de Datos de un Arreglo

Proceso arreglo_numeros

Dimensión números[5];

Para i<-1 Hasta 5 Con Paso 1 Hacer

Escribir "Digite el nro";
Leer números[i];

FinPara

Para i<-1 Hasta 5 Con Paso 1 Hacer

Escribir "El elemento en la posición",i, "es:", números[i];

FinPara

FinProceso

Ejemplos Explicativos:

Vector en PseInt :

```
1
2  Algoritmo EjemploVector
3      Definir notas Como Entero
4      Dimension notas[5]
5
6      Para i <- 1 Hasta 5 Con Paso 1 Hacer
7          Escribir "Ingrese la nota ", i, ": "
8          Leer notas[i]
9      FinPara
10
11     Escribir "Notas ingresadas:"
12     Para i <- 1 Hasta 5 Con Paso 1 Hacer
13         Escribir notas[i]
14     FinPara
15 FinAlgoritmo
16
17 |
```

PSeInt - Ejecutando proceso EJEMPLOVECTOR

*** Ejecución Iniciada. ***

Ingrese la nota 1:

> 5

Ingrese la nota 2:

> 3

Ingrese la nota 3:

> 3

Ingrese la nota 4:

> 2

Ingrese la nota 5:

> 5

Notas ingresadas:

5

3

3

2

5

*** Ejecución Finalizada ***

Vector en C++

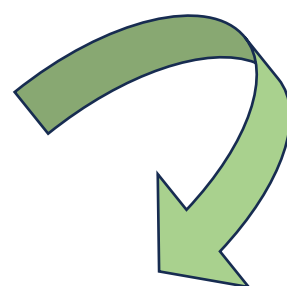
Sirve para evitar que tengas que escribir el prefijo `std::` cada vez que uses elementos de la biblioteca estándar de C++.

- Con `using namespace std;` puedes escribir:

`cout << "Hola mundo";`

- Sin esa línea, deberías escribir:

`std::cout << "Hola mundo";`



```
prueba_vector.cpp
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int notas[5];
6      for(int i = 0; i < 5; i++) {
7          cout << "Ingrese la nota " << i+1 << ": ";
8          cin >> notas[i];
9      }
10     cout << "Notas ingresadas:" << endl;
11     for(int i = 0; i < 5; i++) {
12         cout << notas[i] << endl;
13     }
14     return 0;
15 }
```

```
D:\MGR\AMERICANA\UA\EST x + v
Ingrese la nota 1: 5
Ingrese la nota 2: 3
Ingrese la nota 3: 3
Ingrese la nota 4: 2
Ingrese la nota 5: 5
Notas ingresadas:
5
3
3
2
5
```

Algunas consideraciones a tener en cuenta respecto al código en C++

```
#include <iostream>
```

- Esta línea incluye la librería estándar de entrada y salida en C++.
 - Permite usar cin (entrada) y cout (salida) para interactuar con el usuario.
-

```
using namespace std;
```

- Indica que vamos a usar el espacio de nombres estándar (std) sin necesidad de escribirlo cada vez.
 - Así podemos escribir cout en lugar de std::cout.
-

```
int main() {
```

- Define la función principal del programa.
 - Todo programa en C++ comienza su ejecución en main().
-

```
int notas[5];
```

- Declara un arreglo estático de 5 enteros llamado notas.
 - Cada posición (notas[0] a notas[4]) almacenará una nota ingresada por el usuario
-


```
for(int i = 0; i < 5; i++) {  
    cout << "Ingrese la nota " << i+1 << ": ";  
    cin >> notas[i];  
}
```

- Este es un bucle for que se repite 5 veces.
 - `int i = 0` → inicializa el contador en 0.
 - `i < 5` → condición: mientras i sea menor que 5, el ciclo continúa.
 - `i++` → incrementa el contador en 1 cada vez.
 - Dentro del ciclo:
 - `cout << "Ingrese la nota " << i+1 << ": ";` → muestra un mensaje solicitando la nota número i+1 (para que el usuario vea 1, 2, 3... en lugar de 0, 1, 2...).
 - `cin >> notas[i];` → guarda la nota ingresada en la posición correspondiente del arreglo.
-

```
cout << "Notas ingresadas:" << endl;
```

- Muestra un encabezado en pantalla para indicar que se van a imprimir las notas.
 - `endl` genera un salto de línea.
-

```
for(int i = 0; i < 5; i++) {  
    cout << notas[i] << endl;  
}
```

- Otro bucle for, también de 5 repeticiones.
 - Recorre el arreglo notas desde la posición 0 hasta la 4.
 - `cout << notas[i] << endl;` → imprime cada nota en una línea distinta.
-

```
return 0;
```

`return 0;` indica que el programa terminó correctamente

Ejercicio de clase para practicar

Nº1) Realiza la declaración de un vector de 5 elementos, carga los valores numéricos e imprímelos en pantalla

Solución de PseInt:

```
Algoritmo Vector5Elementos
Definir numeros Como Entero
Dimension numeros[5]

Para i ← 1 Hasta 5 Con Paso 1 Hacer
    Escribir "Ingrese el número ", i, ": "
    Leer numeros[i]
FinPara

Escribir "Los números ingresados son:"
Para i ← 1 Hasta 5 Con Paso 1 Hacer
    Escribir numeros[i]
FinPara
FinAlgoritmo
```



```
#include <iostream>
using namespace std;

int main() {
    int numeros[5]; // Declaración del vector de 5 elementos

    // Carga de valores
    for(int i = 0; i < 5; i++) {
        cout << "Ingrese el número " << i+1 << ": ";
        cin >> numeros[i];
    }

    // Impresión de valores
    cout << "Los números ingresados son:" << endl;
    for(int i = 0; i < 5; i++) {
        cout << numeros[i] << endl;
    }

    return 0;
}
```

Nº 2) Promedio de notas: Leer las notas de 10 estudiantes, almacenarlas en un vector y calcular el promedio.

```
Algoritmo PromedioNotas
  Definir notas Como Entero
  Definir suma, promedio Como Real
  Dimension notas[10]
  suma ← 0

  Para i ← 1 Hasta 10 Con Paso 1 Hacer
    Escribir "Ingrese la nota ", i, ": "
    Leer notas[i]
    suma ← suma + notas[i]
  FinPara

  promedio ← suma / 10
  Escribir "El promedio de las notas es: ", promedio
FinAlgoritmo
```



```
#include <iostream>
using namespace std;

int main() {
    int notas[10];
    int suma = 0;
    float promedio;

    for(int i = 0; i < 10; i++) {
        cout << "Ingrese la nota " << i+1 << ": ";
        cin >> notas[i];
        suma += notas[i];
    }

    promedio = suma / 10.0;
    cout << "El promedio de las notas es: " << promedio << endl;

    return 0;
}
```

Nº 3) Buscar un elemento : Leer 8 números enteros en un vector y verificar si un número dado por el usuario está presente

```
Algoritmo BuscarElemento
  Definir numeros Como Entero
  Definir buscado Como Entero
  Definir encontrado Como Logico
  Dimension numeros[8]
  encontrado ← Falso

  Para i ← 1 Hasta 8 Con Paso 1 Hacer
    Escribir "Ingrese el número ", i, ": "
    Leer numeros[i]
  FinPara

  Escribir "Ingrese el número a buscar: "
  Leer buscado

  Para i ← 1 Hasta 8 Con Paso 1 Hacer
    Si numeros[i] = buscado Entonces
      encontrado ← Verdadero
    FinSi
  FinPara

  Si encontrado Entonces
    Escribir "El número está en el vector."
  SiNo
    Escribir "El número NO está en el vector."
  FinSi
FinAlgoritmo
```



```
#include <iostream>
using namespace std;

int main() {
  int numeros[8];
  int buscado;
  bool encontrado = false;

  for(int i = 0; i < 8; i++) {
    cout << "Ingrese el número " << i+1 << ": ";
    cin >> numeros[i];
  }

  cout << "Ingrese el número a buscar: ";
  cin >> buscado;

  for(int i = 0; i < 8; i++) {
    if(numeros[i] == buscado) {
      encontrado = true;
    }
  }

  if(encontrado)
    cout << "El número está en el vector." << endl;
  else
    cout << "El número NO está en el vector." << endl;

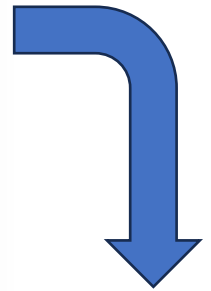
  return 0;
}
```

Nº 4) Matriz de calificaciones: Crear una matriz de 3x3 con calificaciones de estudiantes en tres materias. Mostrar la matriz y calcular el promedio por fila (estudiante).

```
Algoritmo MatrizCalificaciones
Definir matriz Como Entero
Definir suma, promedio Como Real
Dimension matriz[3,3]

Para i ← 1 Hasta 3 Con Paso 1 Hacer
    Para j ← 1 Hasta 3 Con Paso 1 Hacer
        Escribir "Ingrese la nota del estudiante ", i, " en materia ", j
        Leer matriz[i,j]
    FinPara
FinPara

Para i ← 1 Hasta 3 Con Paso 1 Hacer
    suma ← 0
    Para j ← 1 Hasta 3 Con Paso 1 Hacer
        suma ← suma + matriz[i,j]
    FinPara
    promedio ← suma / 3
    Escribir "Promedio del estudiante ", i, ": ", promedio
FinPara
FinAlgoritmo
```



```
#include <iostream>
using namespace std;

int main() {
    int matriz[3][3];
    int suma;
    float promedio;

    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) {
            cout << "Ingrese la nota del estudiante " << i+1
                  << " en materia " << j+1 << ": ";
            cin >> matriz[i][j];
        }
    }

    for(int i = 0; i < 3; i++) {
        suma = 0;
        for(int j = 0; j < 3; j++) {
            suma += matriz[i][j];
        }
        promedio = suma / 3.0;
        cout << "Promedio del estudiante " << i+1 << ": " << promedio << endl;
    }

    return 0;
}
```

Ejercicio de aplicación : Mostrar la solución en C++

- 1) Declarar un vector con 5 productos, ingresar sus precios y calcular el precio total de la compra

- 2) Declarar una matriz de 3x4 que represente el número de pacientes atendidos en 3 salas durante 4 días. Ingresar los datos y calcular el total de pacientes atendidos por cada sala.